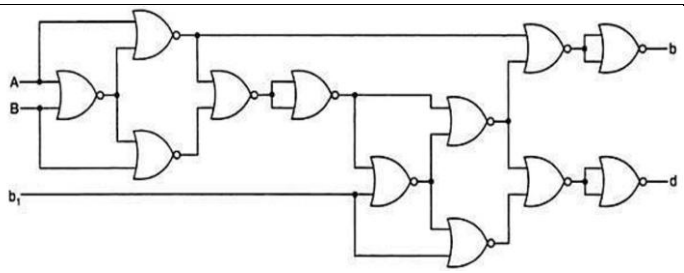
MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SEVENTH SEMESTER BTECH. (E & C) DEGREE END SEMESTER EXAMINATION
NOV/DEC-2024
SUBJECT: (ECE 4062), SET-2

TIME: 3 HOURS **MAX. MARKS: 50**

Instructions to candidates

- Answer ALL questions.
- Missing data may be suitably assumed.

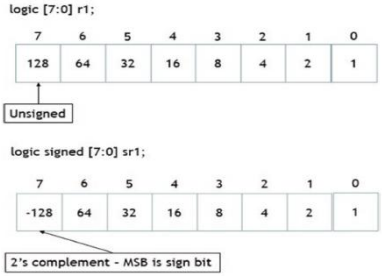
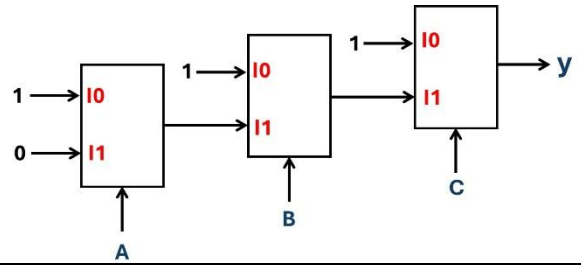
Q. No.	Description	Marks	Course Outcome (1-5)	Competency Levels (1-6)
1 (A)	 Using Boolean algebra, demonstrate that the given circuit is functionally equivalent to a Full substractor block. →The circuit is a full substractor implemented with only NOR gate. b = A xor B xor b_1 d = b_1 (A xor B)' + A'B	**4**	**1**	**3**

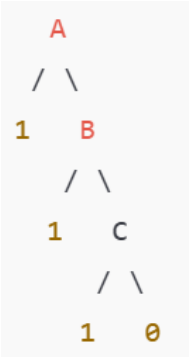
1 (B)	Do you think the following two declarations are equivalent?
	bit [7:0] aBit;
	byte bByte;
	Justify your answer?
	
	Answer is *no* because:
	bit [7:0] aBit; // = 0 to 255
	byte bByte; // = -128 to 127
		3	**2**	**3**

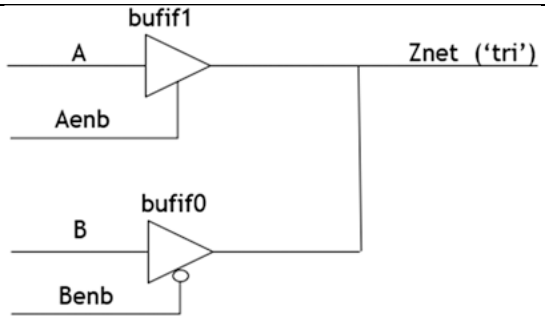
1 (C)	Write the testbench code for 2 to 1 multiplexer by making use of fork join.	**3**	**2**	**3**

	module mux2to1 (
	input logic a, // Input 0
	input logic b, // Input 1
	input logic sel, // Select line
	output logic y // Output
	);
	always_comb begin
	fork
	if (!sel) begin
	y = a; // If sel is 0, assign 'a' to 'y'
	end
	else begin
	y = b; // If sel is 1, assign 'b' to 'y'
	end
	join
	end
	endmodule			

2 (A)	The following code is written to modify the address according to data and an identifier. Write the expected outcome after execution of the given code?
	module add_data;
	int data, address;
	initial begin
	data=1;
	for(int i=0;i<8;i++) begin
	if((i > 2) && (i < 7) && (address<5))begin
	$display("\t\tCalling continue,");
	address= data +i;
	data++;
	continue;
	end
	$display("\t\tAfter Continue\t:: Value of i=%0d",i++);
	$display("\t\tThe address is\t:: address=%0d",address);
	end
	end
	endmodule
	
	→ After Continue ::Value of i=0
	The address is :: address=0
	After Continue :: Value of i=2
	The address is :: address=0
	Calling continue,
	After Continue :: Value of i=5
	The address is :: address=5
	After Continue :: Value of i=7
	The address is :: address=5
		4	**3**	**4**

2 (B)	How are signed and unsigned variables interpreted in SystemVerilog? Give an example. →"r1" is declared as default unsigned 8-bit vector, while "sr1" is declared as signed 8-bit vector. When we assign r1 = −2, since "r1" is unsigned, it will have the value 254 (decimal equivalent of −2). But "sr1" will evaluate as −2 since it is signed. When we add a 1 to "r1," it evaluates to 255 (254 + 1). But when we add a 1 to "sr1," it will be −1 (−2 + 1). logic [7:0] r1; 	**3**	**3**	**2**
2 (C)	What do you mean by shallow and deep copy? How they are defined in SystemVerilog. Shallow copy allocates the memory, copies the variable values and returns the memory handle. In shallow copy, All of the variables are copied across: integers, strings, instance handles, etc. Objects will not be copied, only their handles will be copied. SystemVerilog deep copy copies all the class members and its nested class members. Unlike in shallow copy, only nested class handles will be copied. In shallow copy, Objects will not be copied, only their handles will be copied. To perform a full or deep copy, the custom method needs to be added. In the custom method, a new object is created, all the class properties will be copied to a new handle and the new handle will be returned.	**3**	**3**	**3**
3 (A)	Draw the Reduced Ordered Binary Decision Diagram (ROBDD) for the function y corresponding to the circuit in the figure, following the variable order A<B<C. 	**4**	**2**	**4**

	• The variable order is $A < B < C$. • Starting from the truth table, build a decision tree for $y = \overline{A \cdot B \cdot C}$. **Initial Decision Tree:** • At $A = 0$: $y = 1$ regardless of B and C. • At $A = 1$: • $B = 0$: $y = 1$ regardless of C. • $B = 1$: • $C = 0$: $y = 1$. • $C = 1$: $y = 0$. 			
3 (B)	Draw the BDD for 2 to 1 MUX and represent it using ITE algorithm. **Step-by-Step Construction:** 1. **Root Node:** sel • If $sel = 0$, the output depends only on a. • If $sel = 1$, the output depends only on b. 2. **For $sel = 0$:** • If $a = 0$, $y = 0$. • If $a = 1$, $y = 1$. 3. **For $sel = 1$:** • If $b = 0$, $y = 0$. • If $b = 1$, $y = 1$. The **ITE (If-Then-Else)** representation uses the form: $$ITE(f, g, h) = \text{if } f \text{ then } g \text{ else } h$$ For the 2-to-1 multiplexer: 1. Start at the root node (sel): • If $sel = 1$, the output is determined by b. • If $sel = 0$, the output is determined by a. Thus: $$y = ITE(sel, b, a)$$ This is the compact and correct representation of the multiplexer using IT	**3**	**2**	**4**
3 (C)	What is tristate logic? How is it defined in SystemVerilog. Explain with an example.	**3**	**3**	**3**

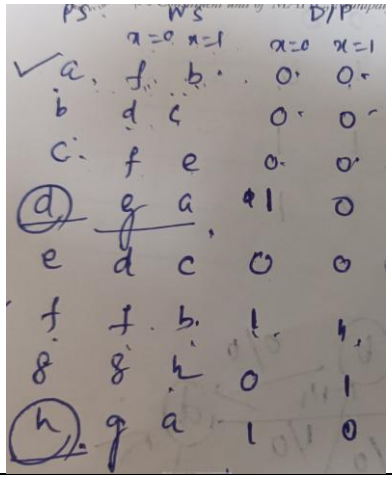


In Fig. 2.1, we show a typical use of "tri" net. There are two drivers, namely, buif0 and bufif1. They both drive a single "tri" net called Znet. Bufif1 drives the net "A" when Aenb is "1," and bufif0 drives the net "B" when Benb is "0." So, when "A" = 1 and "B" = 0, there will be conflict on the net Znet, and the result will be "X" (unknown). But when "A" = 0 and "B" = 1, both drivers will be disabled and Znet will be equal to "Z." In other words, Znet is a resolved type net.

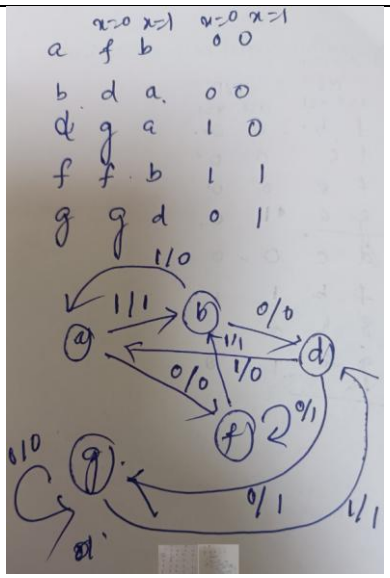
4 (A)

Reduce the number of states in the following state table and tabulate the reduced state table. Draw the state diagram from the reduced state table.

Present State	**Next State**		**Output**	
	***x* = 0**	***x* = 1**	***x* = 0**	***x* = 1**
a	*f*	*b*	0	0
b	*d*	*c*	0	0
c	*f*	*e*	0	0
d	*g*	*a*	1	0
e	*d*	*c*	0	0
f	*f*	*b*	1	1
g	*g*	*h*	0	1
h	*g*	*a*	1	0



4 | **2** | **4**

4 (B)	A test code is written for multiple parallel process execution. Write the compilation output?
	
	module multi_process;
	initial begin
	//fork-1
	fork
	//Process-1
	begin
	$display($time,"\tProcess-1 of fork-1 Started");
	#5;
	$display($time,"\tProcess-1 of fork-1 Finished");
	end
	//Process-2
	begin
	$display($time,"\tProcess-2 of fork-1 Started");
	#20;
	$display($time,"\tProcess-2 of fork-1 Finished");
	end
	join_any
	//fork-2
	fork
	//Process-1
	begin
	$display($time,"\tProcess-1 of fork-2 Started");
	#5;
	$display($time,"\tProcess-1 of fork-2 Finished");
	end
	//Process-2
	begin
	$display($time,"\tProcess-2 of fork-2 Started");
	#20;
		3	**3**	**4**

	$display($time,"\tProcess-2 of fork-2 Finished"); end join_none disable fork; $display($time,"\tAfter disable-fork"); end endmodule 0 Process-1 of fork-1 Started 0 Process-2 of fork-1 Started 5 Process-1 of fork-1 Finished 5 After disable-fork			

4 (C)	What will be the output after the compilation of the following code?
	
	module argument_passing;
	int x,y,z;
	//function to add two integer numbers.
	function int sum(int x,y);
	x = x+y;
	return x+y;
	endfunction
	initial begin
	x = 20;
	y = 30;
	z = sum(x,y);
	$display("\tValue of x = %0d",x);
	$display("\tValue of y = %0d",y);
	$display("\tValue of z = %0d",z);
	end
	endmodule
	
	Value of x = 20
	Value of y = 30
	Value of z = 80
		3	**4**	**4**

5 (A)	There are five processes to access the same resource. The following SystemVerilog code is written to provide synchronization between the processes. What will be the compilation output?
	
	module process_sync;
	semaphore sema;
	initial begin
	sema=new(4);
	fork
	display(2); //process-1
	display(3); //process-2
	display(2); //process-3
	display(1); //process-4
		4	**4**	**4**

	display(1); //process-5
	join
	end
	
	task automatic display(int key);
	sema.get(key);
	$display($time,"\tCurrent Simulation Time, Got %0d keys",key);
	#30;
	sema.put(key); //putting 'key' number of keys to sema
	endtask
	endmodule
	
	0 Current Simulation Time, Got 2 keys
	0 Current Simulation Time, Got 2 keys
	30 Current Simulation Time, Got 3 keys
	30 Current Simulation Time, Got 1 keys
	60 Current Simulation Time, Got 1 keys			

5 (B)	Explain the difference between task and function with proper examples? **Feature** | **Task** | **Function** **Execution** | Executes in **multiple simulation time units** (can use delays). | Executes in **zero simulation time** (purely combinational). **Return Type** | Does not return a value directly; communicates via arguments or internal variables. | Must return a single value using the `return` statement. **Calling Context** | Called using `task_name();`. | Called as part of an expression (e.g., `result = func_name();`). **Usage of Timing Control** | Can contain timing control (e.g., `#delay`, `@event`). | Cannot contain any timing control constructs. **Inputs/Outputs** | Supports **input**, **output**, and **inout** arguments. | Only supports **input** arguments. **Purpose** | Used for complex operations requiring multiple time steps or side effects. | Used for computations and operations that are completed in zero time.	**3**	**3**	**3**

5 (C)	The test code is written for assigning the addr to 10, data to FF, write to 1, and pkt_type to Good Packet.
	Is the output of the code will do the job? If not, then as a verification engineer, what changes will you make in code?
	
	class packet;
	
	bit [31:0] addr;
	bit [31:0] data;
	bit write;
	string pkt_type;
	
	function new(bit [31:0] addr,data,bit write,string pkt_type);
	addr = addr;
	data = data;
	write = write;
	pkt_type = pkt_type;
		3	3	4

```
  endfunction

  function void display();

    $display("\t addr  = %0h",addr);
    $display("\t data  = %0h",data);
    $display("\t write = %0h",write);
    $display("\t pkt_type  = %0s",pkt_type);
  endfunction

endclass

module assignment;
  packet pkt;
  initial begin
    pkt = new(32'h10,32'hFF,1,"Good Packet");
    pkt.display();
  end

endmodule

 addr  = 0
 data  = 0
 write = 0
 pkt_type  =
```

Use of “this” keyword